DR. RVR NRI INSTITUTE OF TECHNOLOGY

PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL) ASSIGNMENT

CONSOLIDATED REPORT: CASE STUDIES 61–70

Name: V.Rishika

Roll No: 26001431

Branch: CSD

Section: 1

61. Nested List Examination Seating Validation

Problem Statement

Validate a complete seating matrix for duplicate roll numbers, reserved positions, empty seats, and invalid student allocations.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

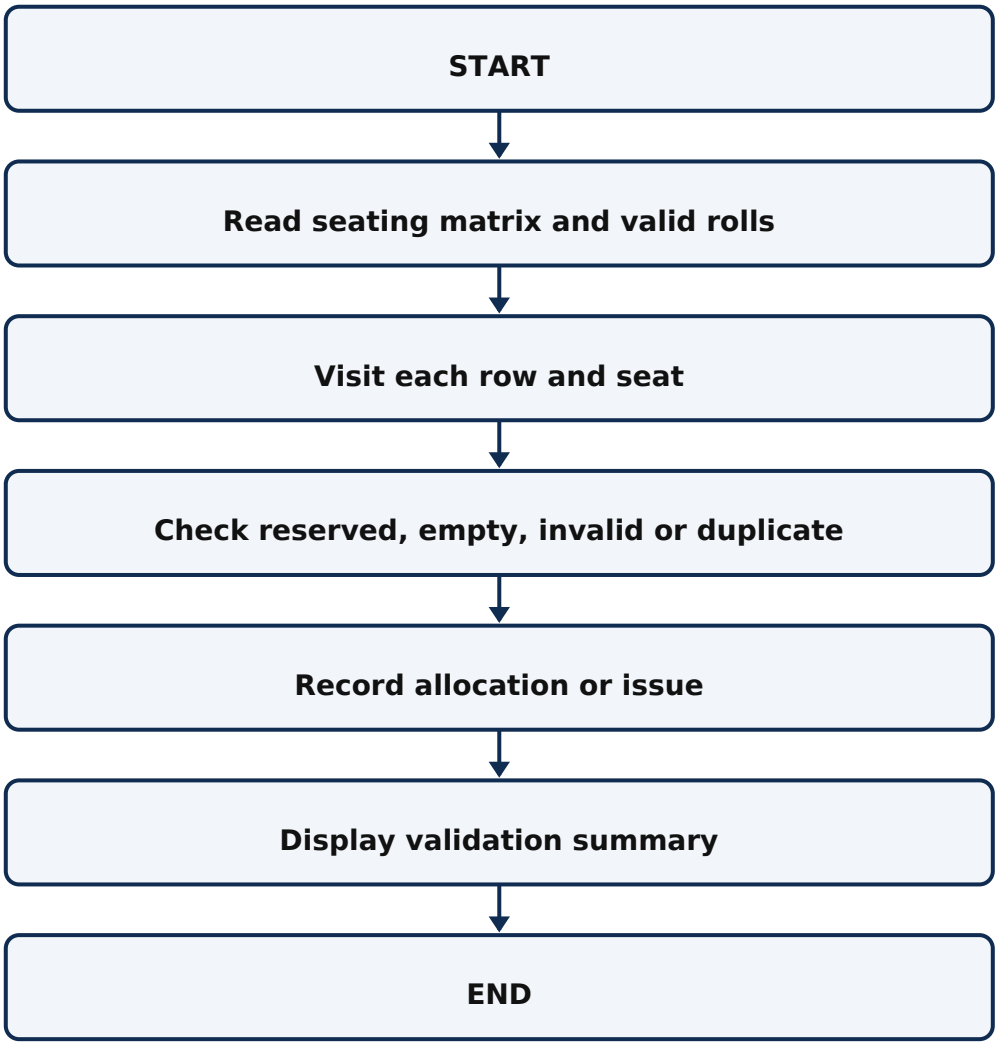
Assumptions

Seats use 0 for empty and R for reserved; valid rolls are supplied in a list.

Algorithm

1. Read seating matrix and valid rolls.
2. Visit each row and seat.
3. Check reserved, empty, invalid or duplicate.
4. Record allocation or issue.
5. Display validation summary.

Flowchart (Connected Boxes)


START
Read seating matrix and valid rolls
Visit each row and seat
Check reserved, empty, invalid or duplicate
Record allocation or issue
Display validation summary
END

Pseudocode

```
BEGIN
    READ SEATING MATRIX AND VALID ROLLS
    VISIT EACH ROW AND SEAT
    CHECK RESERVED, EMPTY, INVALID OR DUPLICATE
    RECORD ALLOCATION OR ISSUE
    DISPLAY VALIDATION SUMMARY
END
```

Python Source Code

```
"""Nested List Examination Seating Validation"""
valid = [101, 102, 103, 104, 105]
seats = [[101, 102, 'R'], [103, 101, 0], [999, 'R', 104]]
seen = []
for i in range(len(seats)):
    for j in range(len(seats[i])):
        x = seats[i][j]
        position = (i + 1, j + 1)
        if x == 'R': print(position, 'Reserved')
        elif x == 0: print(position, 'Empty')
        elif x not in valid: print(position, 'Invalid roll:', x)
        elif x in seen: print(position, 'Duplicate roll:', x)
        else: seen.append(x)
print('Valid unique allocations:', seen)
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
(1, 3) Reserved
(2, 2) Duplicate roll: 101
(2, 3) Empty
(3, 1) Invalid roll: 999
(3, 2) Reserved
Valid unique allocations: [101, 102, 103, 104]
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

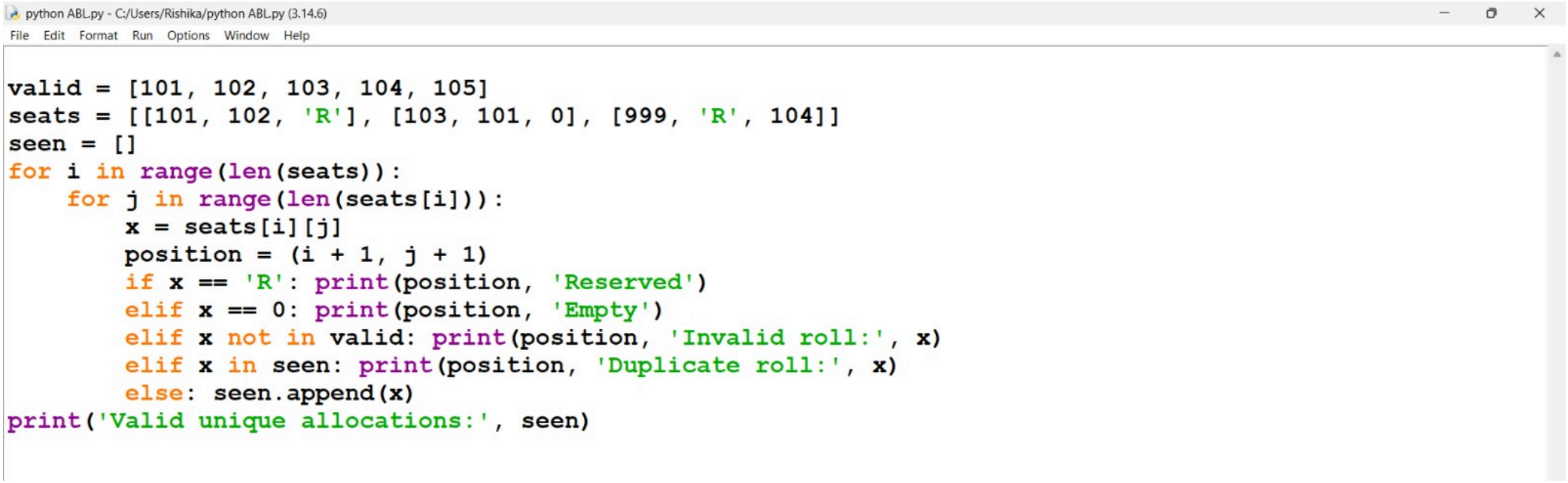
Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

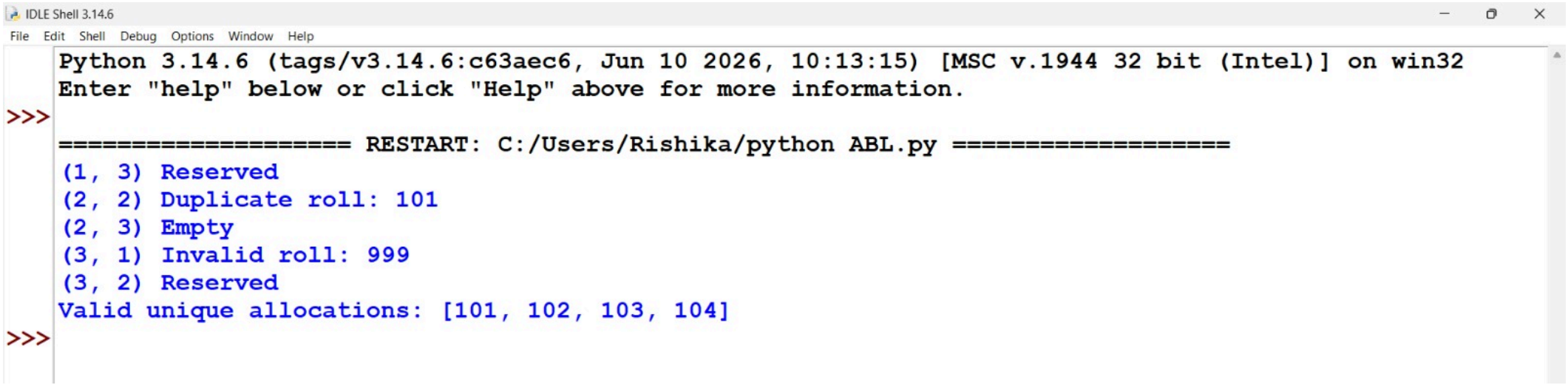
This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

Question 61 - Python IDLE Program Screenshot



Original screenshot supplied by student; cropped to improve text readability.

Question 61 - Python IDLE Output Screenshot



Original screenshot supplied by student; cropped to improve text readability.

62. Water Distribution Fairness Analysis

Problem Statement

Analyse area-wise water supply across several days, calculate shortages against demand, compare distribution levels, and identify the most underserved areas.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

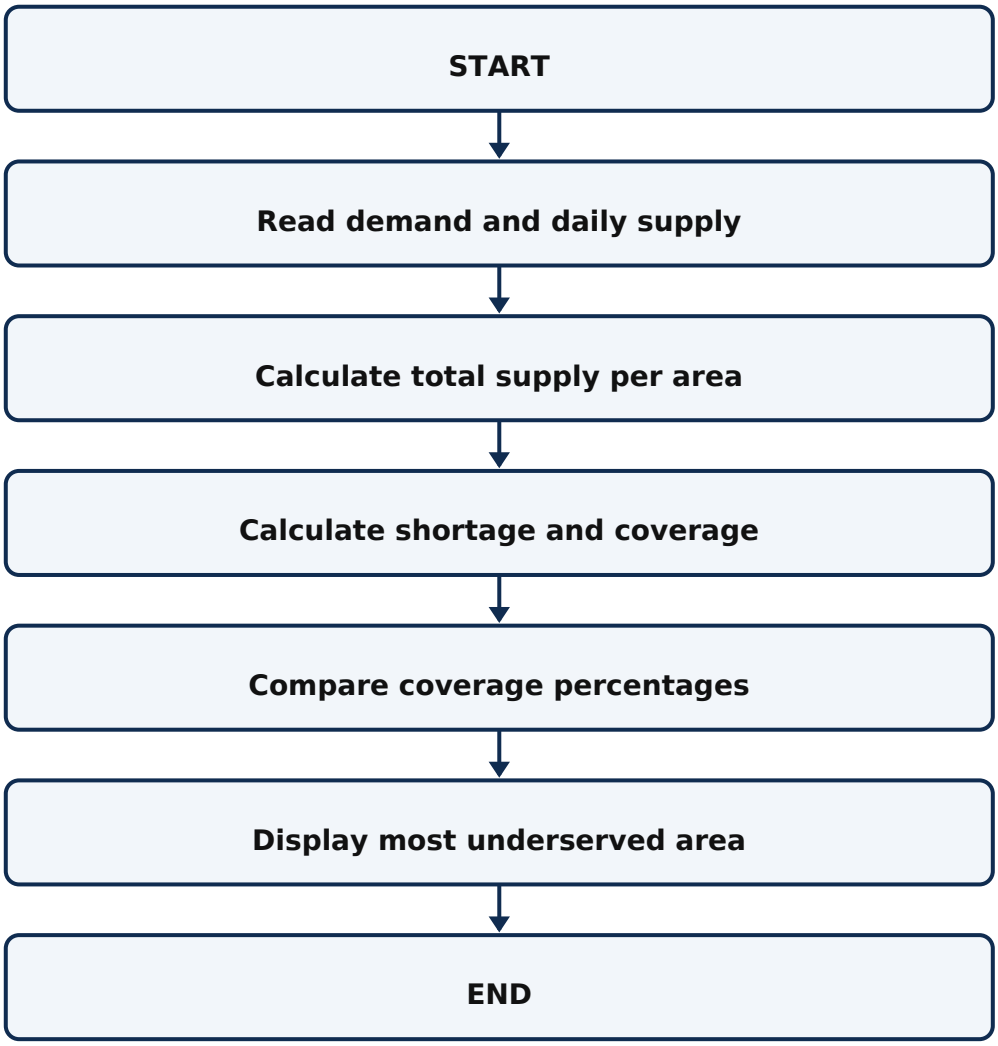
Assumptions

Demand and supply are in kilolitres; demand is daily and nonzero.

Algorithm

1. Read demand and daily supply.
2. Calculate total supply per area.
3. Calculate shortage and coverage.
4. Compare coverage percentages.
5. Display most underserved area.

Flowchart (Connected Boxes)


START
Read demand and daily supply
Calculate total supply per area
Calculate shortage and coverage
Compare coverage percentages
Display most underserved area
END

Pseudocode

```
BEGIN
    READ DEMAND AND DAILY SUPPLY
    CALCULATE TOTAL SUPPLY PER AREA
    CALCULATE SHORTAGE AND COVERAGE
    COMPARE COVERAGE PERCENTAGES
    DISPLAY MOST UNDERSERVED AREA
END
```

Python Source Code

```
"""Water Distribution Fairness Analysis"""
areas = ['North', 'South', 'East']
demand = [100, 80, 120]
supply = [[90, 80, 100], [80, 85, 75], [100, 90, 110]]
results = []
for i in range(len(areas)):
    total = sum(supply[i]); required = demand[i] * len(supply[i])
    shortage = max(0, required - total)
    ratio = total / required * 100
    results.append((areas[i], shortage, ratio))
    print(areas[i], 'Supply:', total, 'Shortage:', shortage, 'Coverage: %.2f%%' % rati
print('Most underserved:', min(results, key=lambda x: x[2])[0])
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
North Supply: 270 Shortage: 30 Coverage: 90.00%
South Supply: 240 Shortage: 0 Coverage: 100.00%
East Supply: 300 Shortage: 60 Coverage: 83.33%
Most underserved: East
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

Question 62 - Python IDLE Program Screenshot

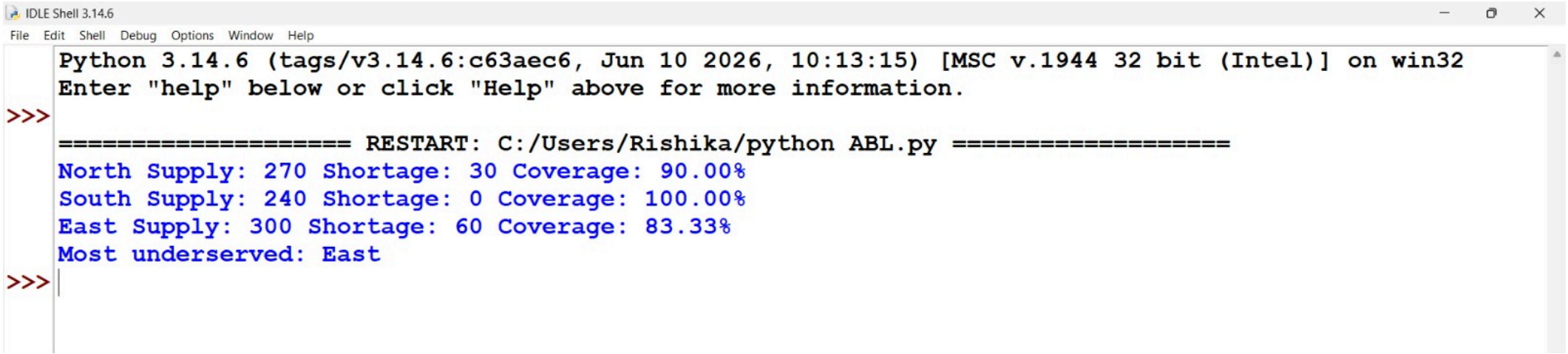
python ABL.py - C:/Users/Rishika/python ABL.py (3.14.6)

File Edit Format Run Options Window Help

```
areas = ['North', 'South', 'East']
demand = [100, 80, 120]
supply = [[90, 80, 100], [80, 85, 75], [100, 90, 110]]
results = []
for i in range(len(areas)):
    total = sum(supply[i]); required = demand[i] * len(supply[i])
    shortage = max(0, required - total)
    ratio = total / required * 100
    results.append((areas[i], shortage, ratio))
    print(areas[i], 'Supply:', total, 'Shortage:', shortage, 'Coverage: %.2f%%' % ratio)
print('Most underserved:', min(results, key=lambda x: x[2])[0])
```

Original screenshot supplied by student; cropped to improve text readability.

Question 62 - Python IDLE Output Screenshot



Original screenshot supplied by student; cropped to improve text readability.

63. Multi-Product Price Fluctuation Analysis

Problem Statement

Process historical product prices stored in lists, calculate percentage changes, identify maximum fluctuations, and flag products exceeding a supplied threshold.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

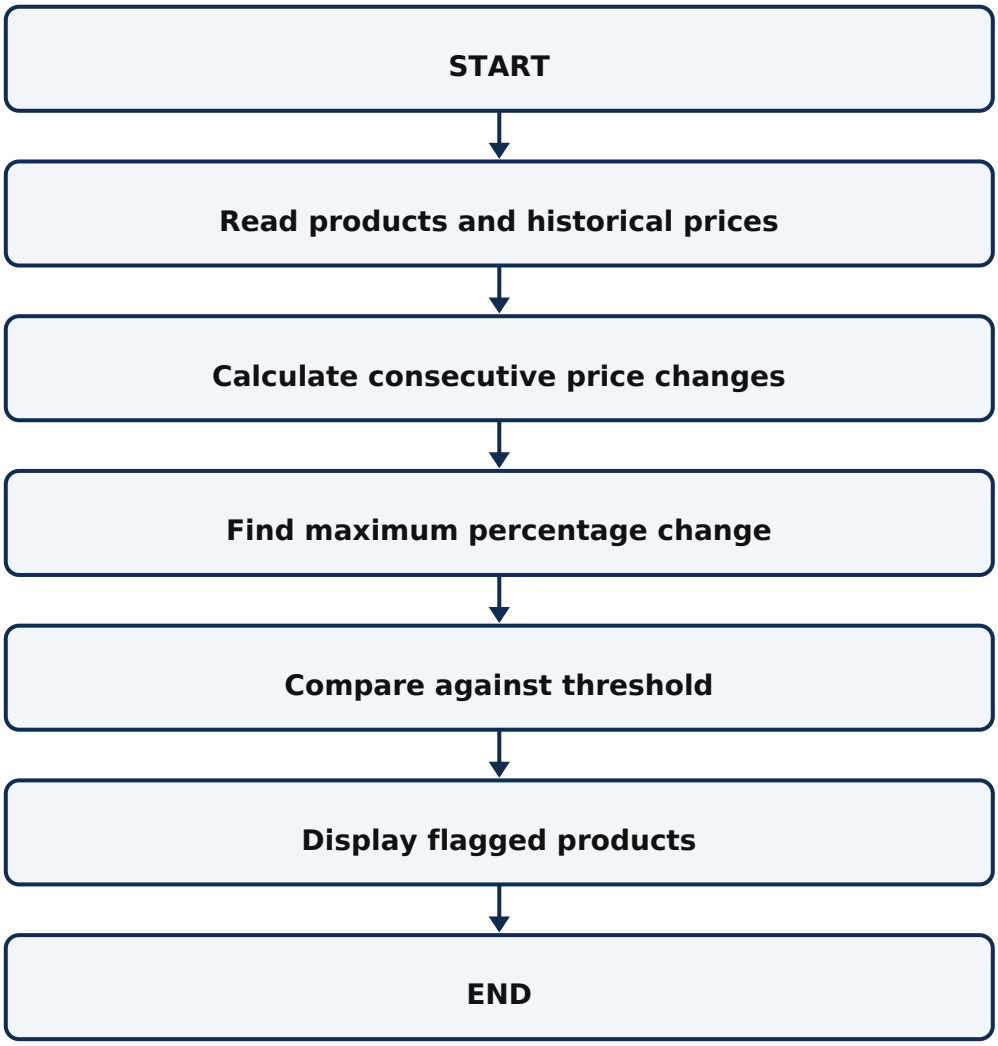
Assumptions

Historical prices are positive; threshold is a percentage.

Algorithm

1. Read products and historical prices.
2. Calculate consecutive price changes.
3. Find maximum percentage change.
4. Compare against threshold.
5. Display flagged products.

Flowchart (Connected Boxes)


START
Read products and historical prices
Calculate consecutive price changes
Find maximum percentage change
Compare against threshold
Display flagged products
END

Pseudocode

```
BEGIN
    READ PRODUCTS AND HISTORICAL PRICES
    CALCULATE CONSECUTIVE PRICE CHANGES
    FIND MAXIMUM PERCENTAGE CHANGE
    COMPARE AGAINST THRESHOLD
    DISPLAY FLAGGED PRODUCTS
END
```

Python Source Code

```
"""Multi-Product Price Fluctuation Analysis"""
prices = {'Rice':[50, 55, 52], 'Oil':[100, 120, 110], 'Sugar':[40, 41, 42]}
threshold = 10
for name, history in prices.items():
    changes = []
    for i in range(1, len(history)):
        changes.append((history[i] - history[i-1]) / history[i-1] * 100)
    largest = max(abs(x) for x in changes)
    print(name, 'Changes:', [round(x, 2) for x in changes], 'Max:', round(largest, 2),
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
Rice Changes: [10.0, -5.45] Max: 10.0 Flag: False
Oil Changes: [20.0, -8.33] Max: 20.0 Flag: True
Sugar Changes: [2.5, 2.44] Max: 2.5 Flag: False
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

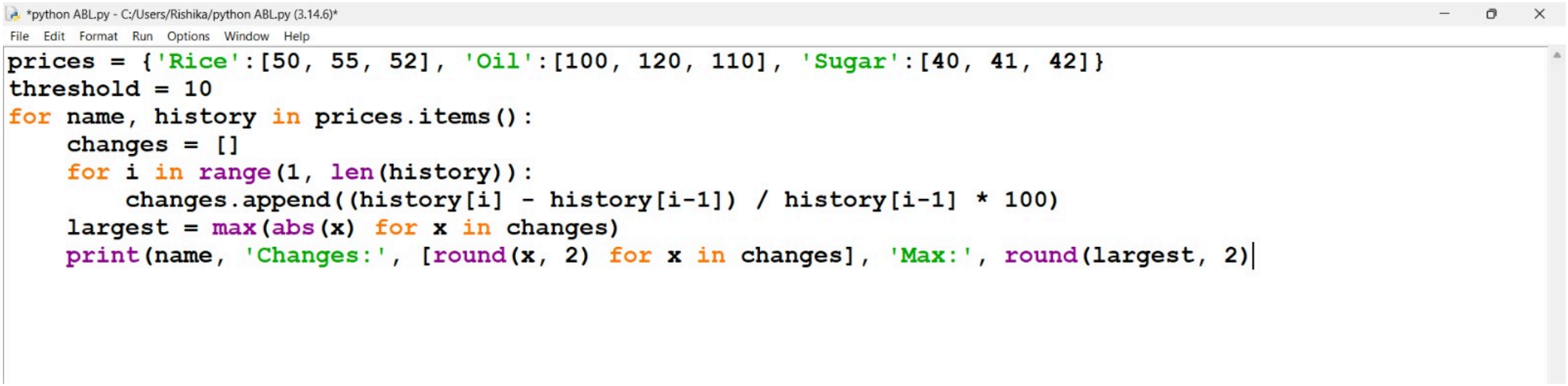
Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

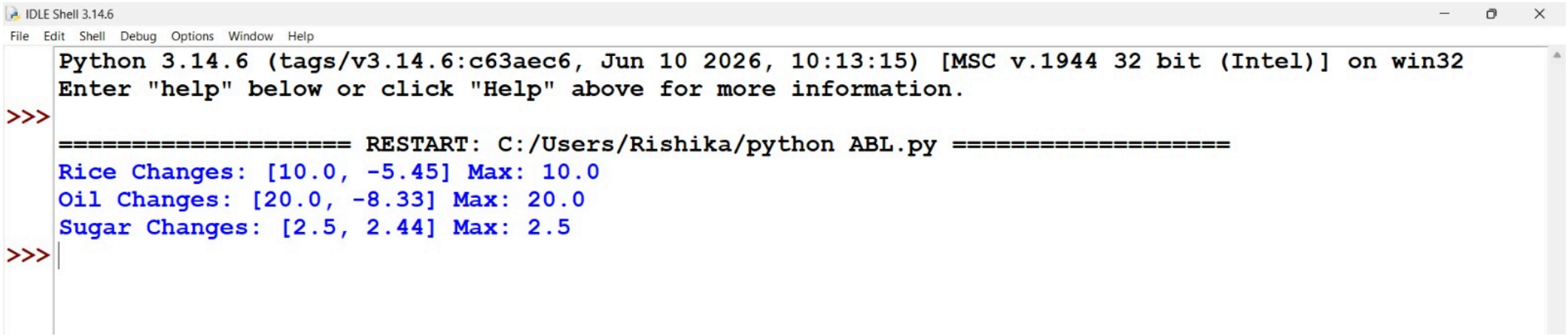
This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

Question 63 - Python IDLE Program Screenshot



Original screenshot supplied by student; cropped to improve text readability.

Question 63 - Python IDLE Output Screenshot



Original screenshot supplied by student; cropped to improve text readability.

64. Tuple-Based Course Registration Validation

Problem Statement

Represent course registrations as tuples, identify duplicate registrations and invalid course combinations, and determine students whose selections violate supplied rules.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

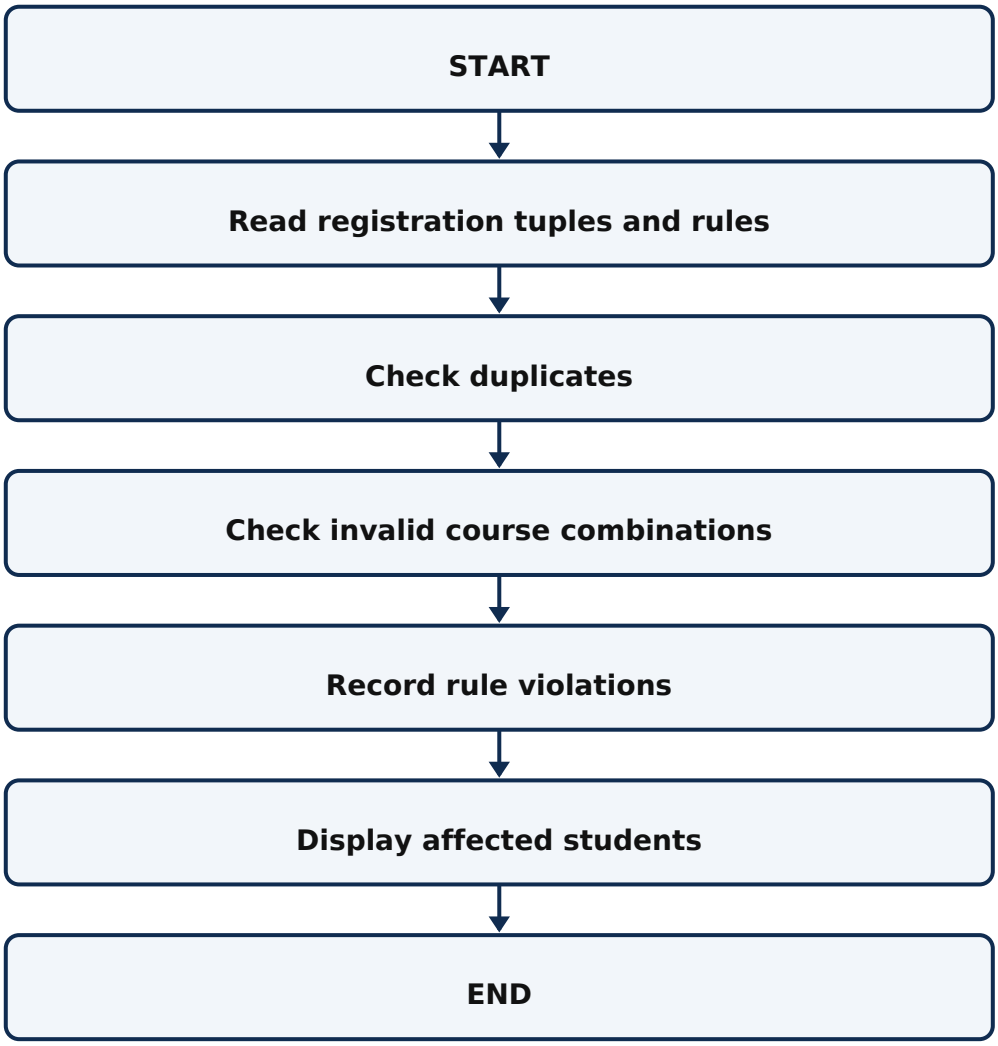
Assumptions

Each registration is a (student, course) tuple; AI101 and AI102 cannot be combined.

Algorithm

1. Read registration tuples and rules.
2. Check duplicates.
3. Check invalid course combinations.
4. Record rule violations.
5. Display affected students.

Flowchart (Connected Boxes)


START
Read registration tuples and rules
Check duplicates
Check invalid course combinations
Record rule violations
Display affected students
END

Pseudocode

```
BEGIN
    READ REGISTRATION TUPLES AND RULES
    CHECK DUPLICATES
    CHECK INVALID COURSE COMBINATIONS
    RECORD RULE VIOLATIONS
    DISPLAY AFFECTED STUDENTS
END
```

Python Source Code

```
"""Tuple-Based Course Registration Validation"""
registrations = [('A','PY101'), ('A','AI101'), ('A','AI102'), ('A','PY101'), ('B','PY1
valid = ('PY101', 'AI101', 'AI102')
seen = []
students = {}
for student, course in registrations:
    if (student, course) in seen: print('Duplicate:', student, course)
    else: seen.append((student, course))
    if course not in valid: print('Invalid course:', student, course)
    if student not in students: students[student] = []
    students[student].append(course)
for student, courses in students.items():
    if 'AI101' in courses and 'AI102' in courses:
        print('Prohibited combination:', student)
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
Duplicate: A PY101
Invalid course: B BAD
Prohibited combination: A
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

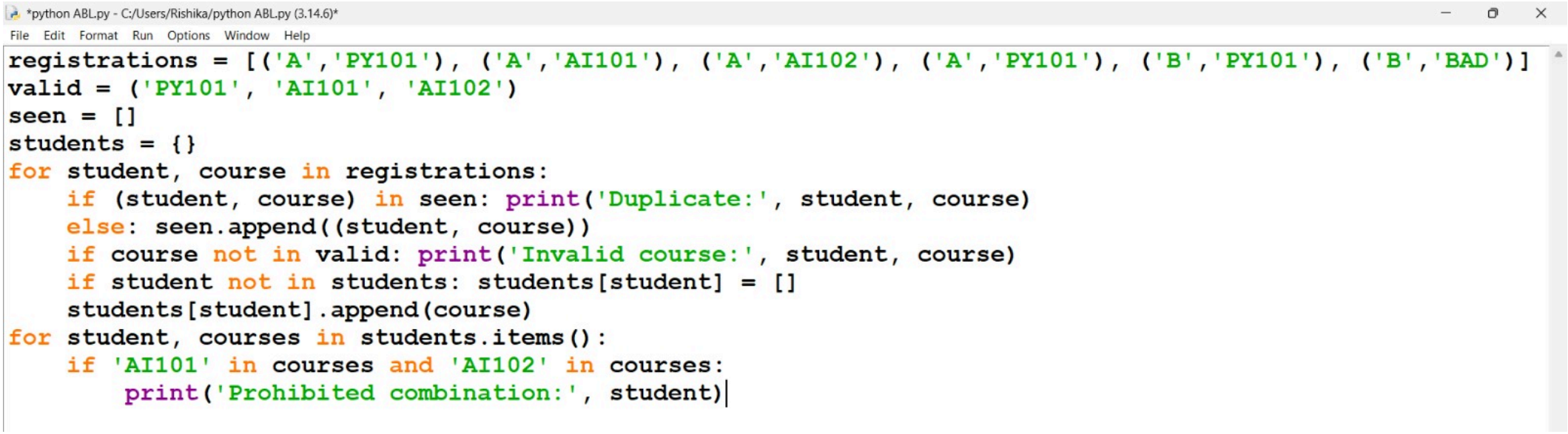
Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

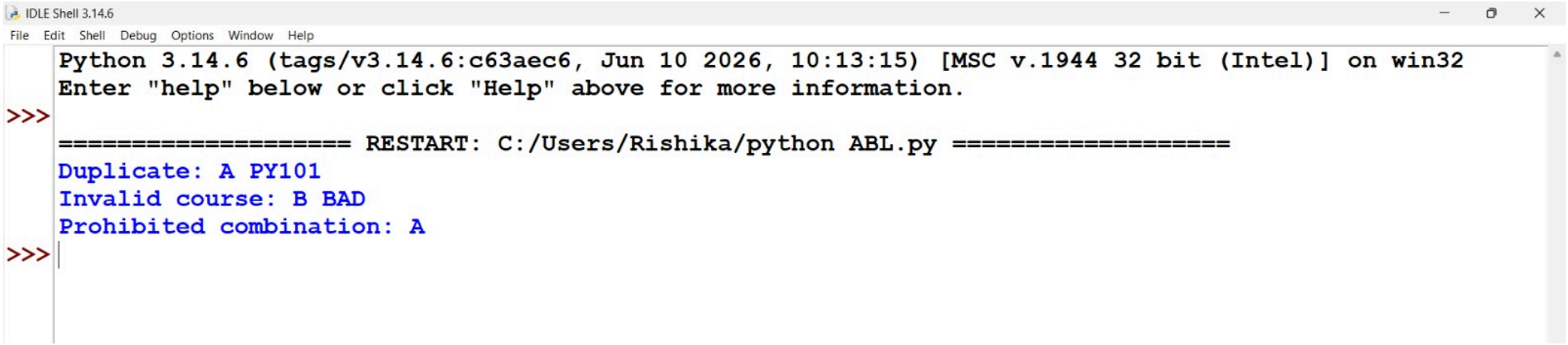
This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

Question 64 - Python IDLE Program Screenshot



Original screenshot supplied by student; cropped to improve text readability.

Question 64 - Python IDLE Output Screenshot



Original screenshot supplied by student; cropped to improve text readability.

65. Campus Resource Allocation Under Constraints

Problem Statement

Compare equipment requirements with available resources across multiple activities, calculate shortages, and produce a feasible allocation using a specified priority order.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

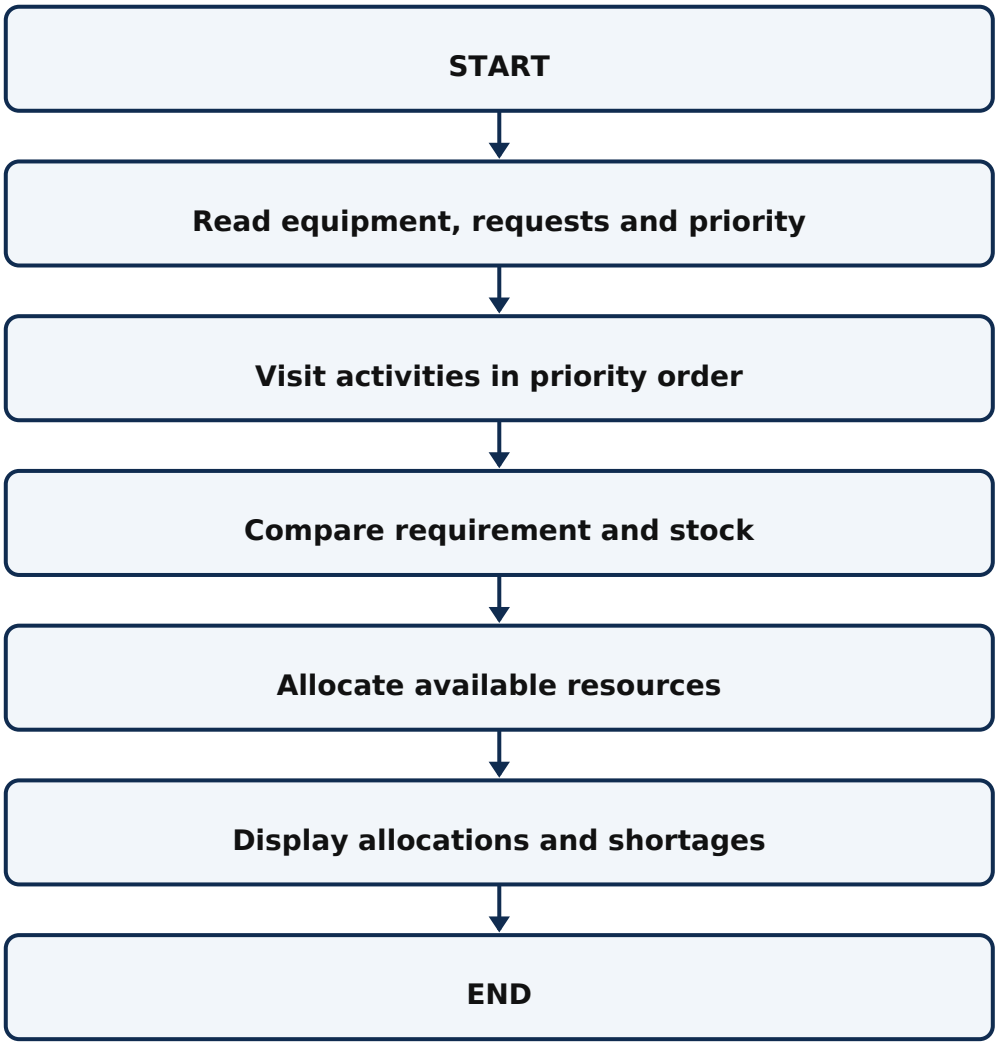
Assumptions

Activities are listed in descending priority; partial allocation is permitted.

Algorithm

1. Read equipment, requests and priority.
2. Visit activities in priority order.
3. Compare requirement and stock.
4. Allocate available resources.
5. Display allocations and shortages.

Flowchart (Connected Boxes)


START
Read equipment, requests and priority
Visit activities in priority order
Compare requirement and stock
Allocate available resources
Display allocations and shortages
END

Pseudocode

```
BEGIN
    READ EQUIPMENT, REQUESTS AND PRIORITY
    VISIT ACTIVITIES IN PRIORITY ORDER
    COMPARE REQUIREMENT AND STOCK
    ALLOCATE AVAILABLE RESOURCES
    DISPLAY ALLOCATIONS AND SHORTAGES
END
```

Python Source Code

```
"""Campus Resource Allocation Under Constraints"""
stock = {'Laptop': 8, 'Projector': 2}
activities = [('Workshop', {'Laptop':5, 'Projector':1}), ('Seminar', {'Laptop':4, 'Pro
for name, needs in activities:
    allocated = {}; shortage = {}
    for item, required in needs.items():
        given = min(required, stock.get(item, 0))
        allocated[item] = given
        shortage[item] = required - given
        stock[item] -= given
    print(name, 'Allocated:', allocated, 'Shortage:', shortage)
print('Remaining:', stock)
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
Workshop Allocated: {'Laptop': 5, 'Projector': 1} Shortage: {'Laptop': 0, 'Projector':
Seminar Allocated: {'Laptop': 3, 'Projector': 1} Shortage: {'Laptop': 1, 'Projector':
Club Allocated: {'Laptop': 0, 'Projector': 0} Shortage: {'Laptop': 2, 'Projector': 1}
Remaining: {'Laptop': 0, 'Projector': 0}
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

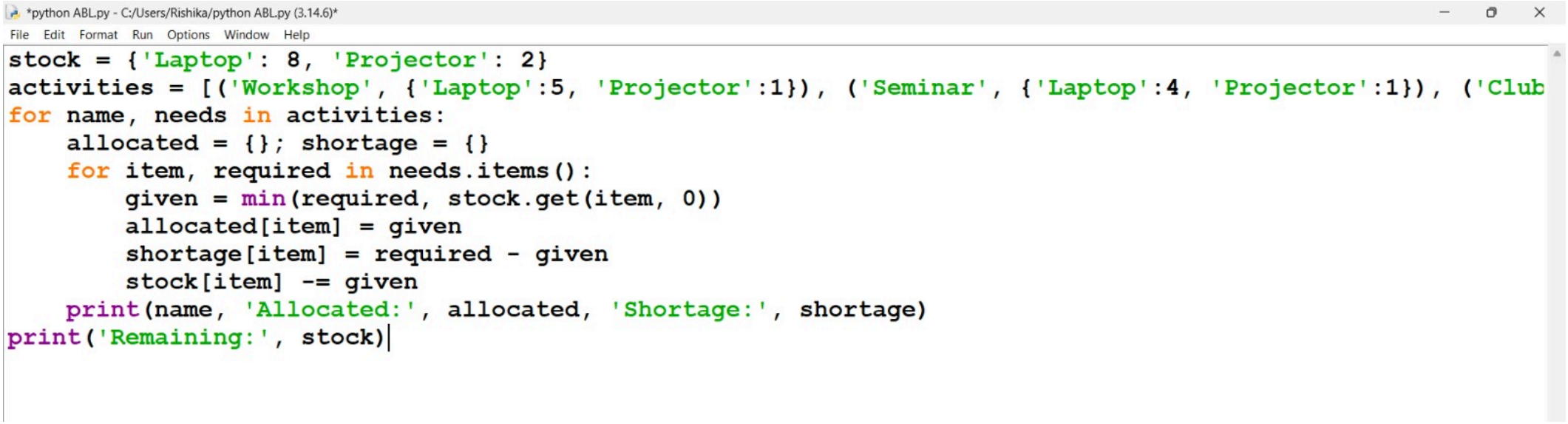
Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

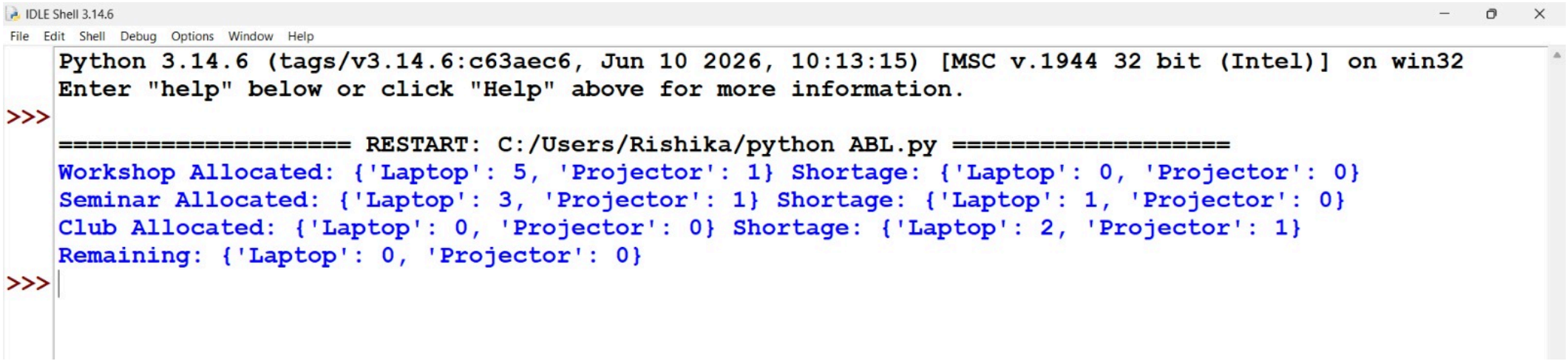
This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

Question 65 - Python IDLE Program Screenshot



Original screenshot supplied by student; cropped to improve text readability.

Question 65 - Python IDLE Output Screenshot



Original screenshot supplied by student; cropped to improve text readability.

66. Student Academic Risk Classification

Problem Statement

Combine lists of marks, attendance, and laboratory scores to classify students into risk categories, identify multiple simultaneous risk factors, and generate a summary.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

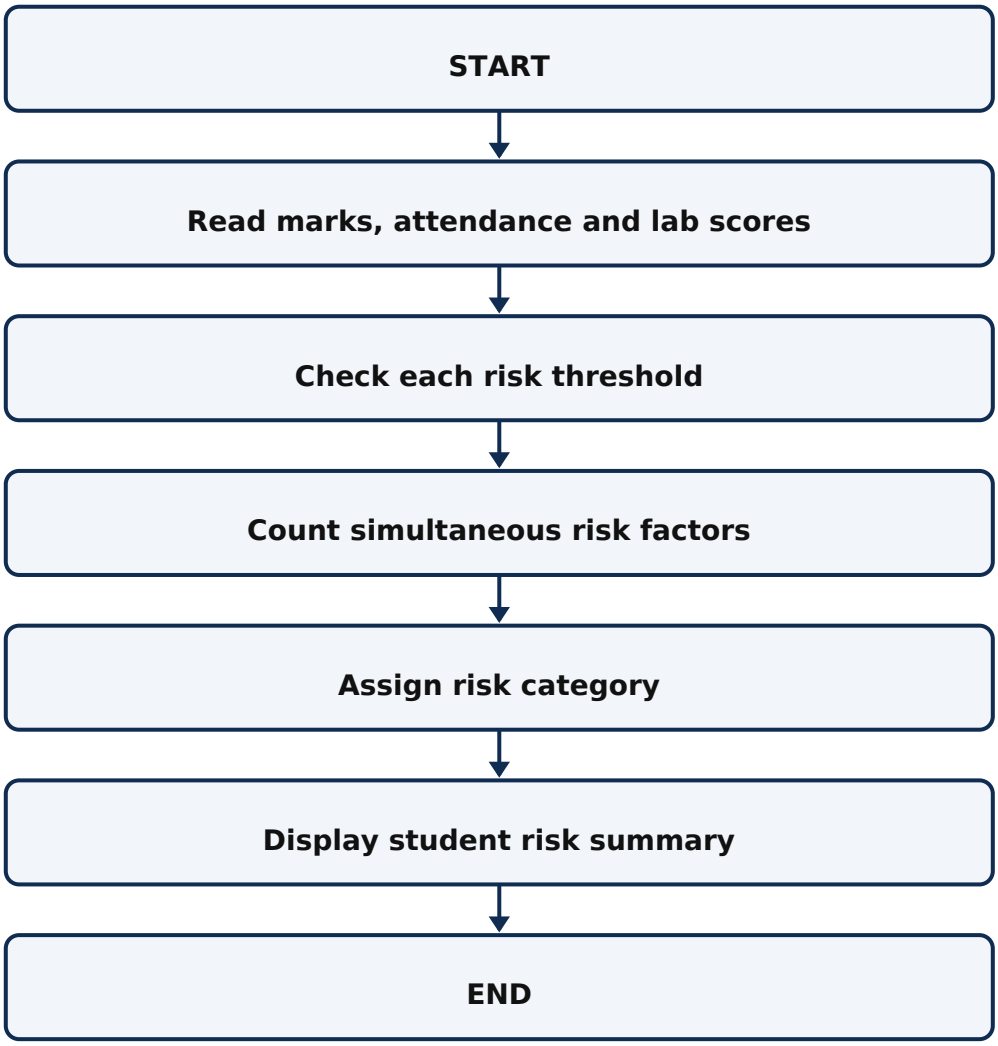
Assumptions

Risk factors are marks below 40, attendance below 75%, and lab score below 50.

Algorithm

1. Read marks, attendance and lab scores.
2. Check each risk threshold.
3. Count simultaneous risk factors.
4. Assign risk category.
5. Display student risk summary.

Flowchart (Connected Boxes)


START
Read marks, attendance and lab scores
Check each risk threshold
Count simultaneous risk factors
Assign risk category
Display student risk summary
END

Pseudocode

```
BEGIN
    READ MARKS, ATTENDANCE AND LAB SCORES
    CHECK EACH RISK THRESHOLD
    COUNT SIMULTANEOUS RISK FACTORS
    ASSIGN RISK CATEGORY
    DISPLAY STUDENT RISK SUMMARY
END
```

Python Source Code

```
"""Student Academic Risk Classification"""
names = ['Anu', 'Bala', 'Charan', 'Divya']
marks = [85, 35, 45, 30]
attendance = [90, 70, 60, 65]
labs = [80, 45, 70, 40]
summary = {'Low':0, 'Moderate':0, 'High':0}
for i in range(len(names)):
    factors = []
    if marks[i] < 40: factors.append('Marks')
    if attendance[i] < 75: factors.append('Attendance')
    if labs[i] < 50: factors.append('Lab')
    category = 'High' if len(factors) >= 2 else ('Moderate' if factors else 'Low')
    summary[category] += 1
    print(names[i], category, factors)
print('Summary:', summary)
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
Anu Low []
Bala High ['Marks', 'Attendance', 'Lab']
Charan Moderate ['Attendance']
Divya High ['Marks', 'Attendance', 'Lab']
Summary: {'Low': 1, 'Moderate': 1, 'High': 2}
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

Expected Result

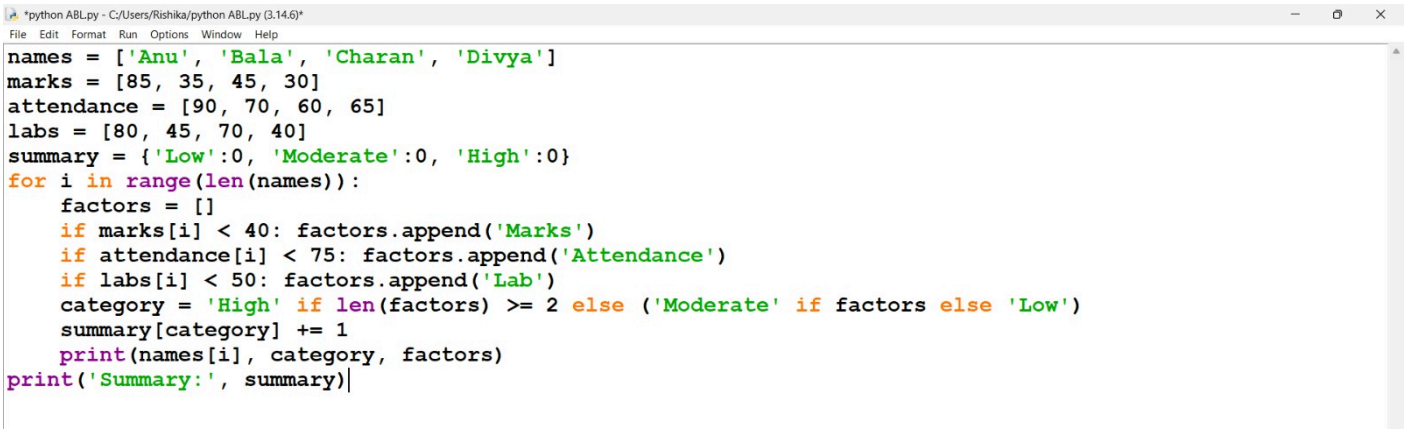
The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

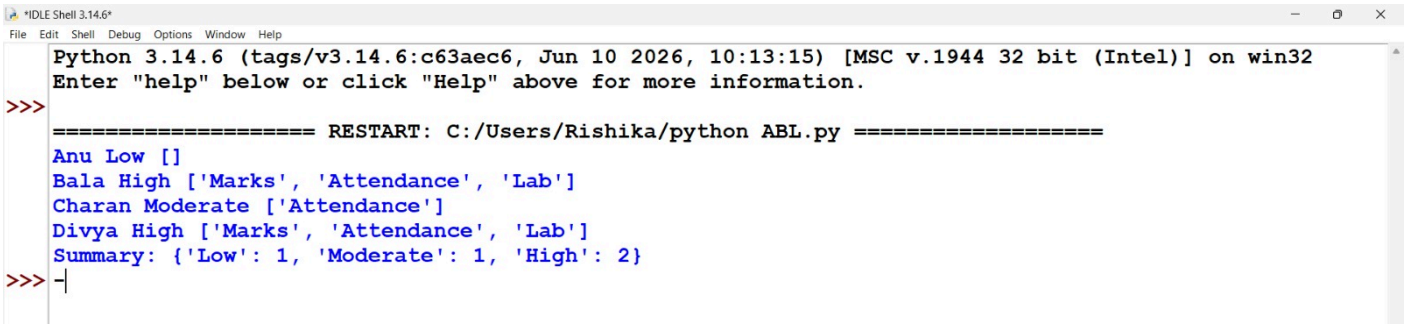
Question 66 - Python IDLE Program Screenshot

Original screenshot supplied by student



Question 66 - Python IDLE Output Screenshot

Original screenshot supplied by student



67. Retail Order Dispatch and Return Analysis

Problem Statement

Process order quantities, dispatch records, and returned quantities; calculate outstanding orders, update inventory, and detect inconsistent records.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

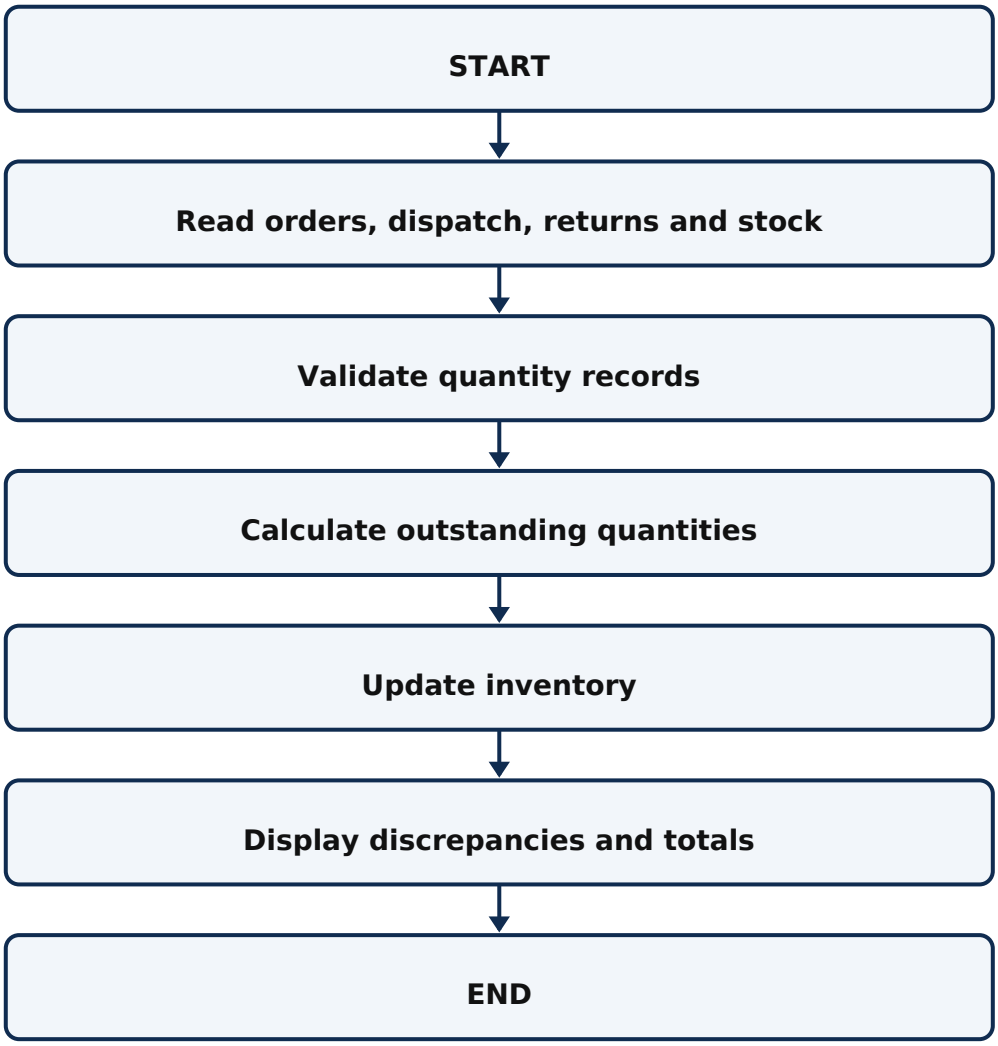
Assumptions

Returned goods are resalable; starting inventory is given.

Algorithm

1. Read orders, dispatch, returns and stock.
2. Validate quantity records.
3. Calculate outstanding quantities.
4. Update inventory.
5. Display discrepancies and totals.

Flowchart (Connected Boxes)


START
Read orders, dispatch, returns and stock
Validate quantity records
Calculate outstanding quantities
Update inventory
Display discrepancies and totals
END

Pseudocode

```
BEGIN
    READ ORDERS, DISPATCH, RETURNS AND STOCK
    VALIDATE QUANTITY RECORDS
    CALCULATE OUTSTANDING QUANTITIES
    UPDATE INVENTORY
    DISPLAY DISCREPANCIES AND TOTALS
END
```

Python Source Code

```
"""Retail Order Dispatch and Return Analysis"""
products = ['Pen', 'Book', 'Bag']
ordered = [20, 10, 5]
dispatched = [15, 12, 4]
returned = [2, 1, 5]
stock = [30, 20, 10]
for i in range(len(products)):
    if dispatched[i] > ordered[i] or returned[i] > dispatched[i] or min(ordered[i], di
        print(products[i], 'Inconsistent record')
        continue
    if dispatched[i] > stock[i]:
        print(products[i], 'Insufficient stock')
        continue
    pending = ordered[i] - dispatched[i]
    stock[i] = stock[i] - dispatched[i] + returned[i]
    print(products[i], 'Pending:', pending, 'Updated stock:', stock[i])
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
Pen Pending: 5 Updated stock: 17
Book Inconsistent record
Bag Inconsistent record
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

68. Energy Conservation Before-and-After Analysis

Problem Statement

Compare appliance-wise electricity usage before and after an intervention, calculate savings percentages, and rank appliances by achieved savings.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

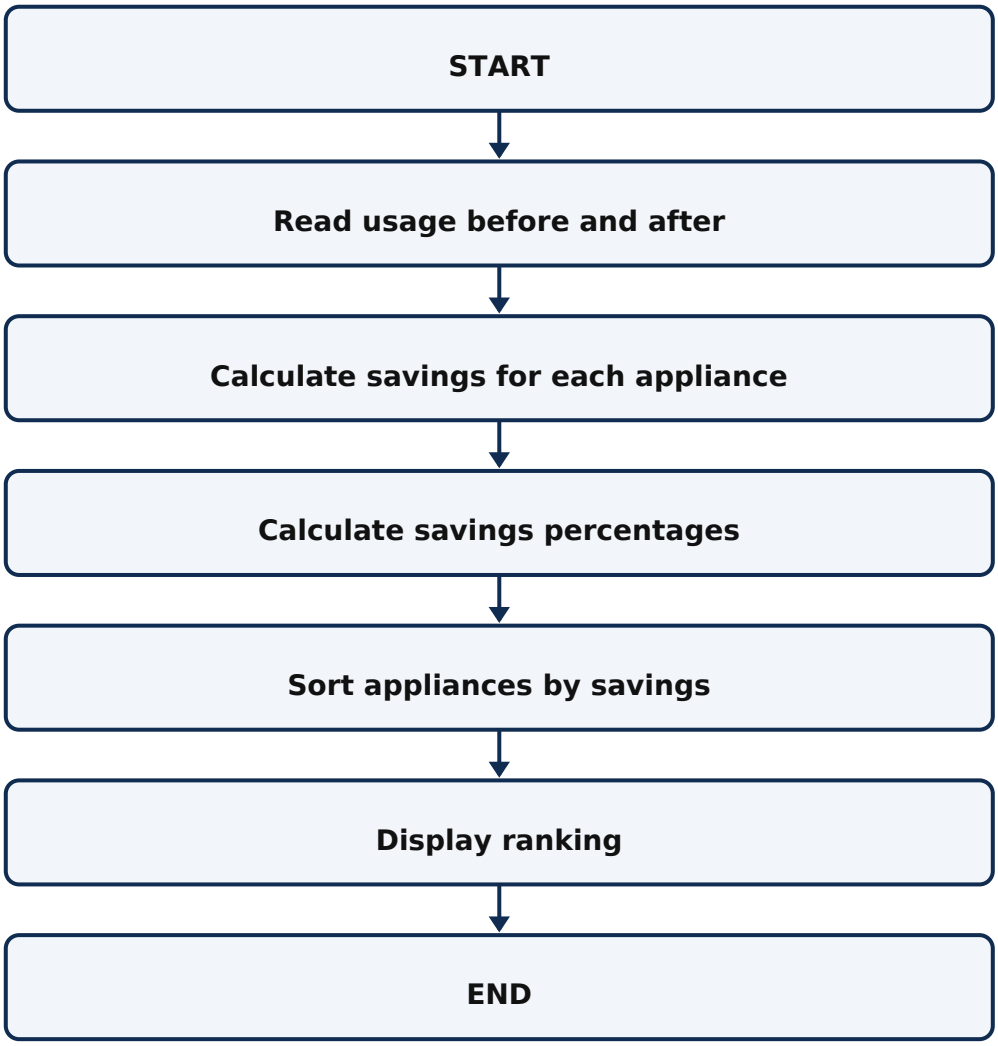
Assumptions

Usage is measured over equal periods; before values are positive.

Algorithm

1. Read usage before and after.
2. Calculate savings for each appliance.
3. Calculate savings percentages.
4. Sort appliances by savings.
5. Display ranking.

Flowchart (Connected Boxes)


START
Read usage before and after
Calculate savings for each appliance
Calculate savings percentages
Sort appliances by savings
Display ranking
END

Pseudocode

```
BEGIN
    READ USAGE BEFORE AND AFTER
    CALCULATE SAVINGS FOR EACH APPLIANCE
    CALCULATE SAVINGS PERCENTAGES
    SORT APPLIANCES BY SAVINGS
    DISPLAY RANKING
END
```

Python Source Code

```
"""Energy Conservation Before-and-After Analysis"""
names = ['AC', 'Fan', 'Light']
before = [200, 100, 50]
after = [150, 90, 30]
results = []
for i in range(len(names)):
    saved = before[i] - after[i]
    percent = saved / before[i] * 100
    results.append((names[i], percent))
    print(names[i], 'Saved:', saved, 'Percentage: %.2f%%' % percent)
print('Ranking:', sorted(results, key=lambda x: (-x[1], x[0])))
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
AC Saved: 50 Percentage: 25.00%
Fan Saved: 10 Percentage: 10.00%
Light Saved: 20 Percentage: 40.00%
Ranking: [('Light', 40.0), ('AC', 25.0), ('Fan', 10.0)]
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

69. Nested List Sports Performance Analysis

Problem Statement

Analyse players’ scores across multiple matches, calculate consistency measures, identify performance trends, and rank players under supplied tiebreaking rules.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

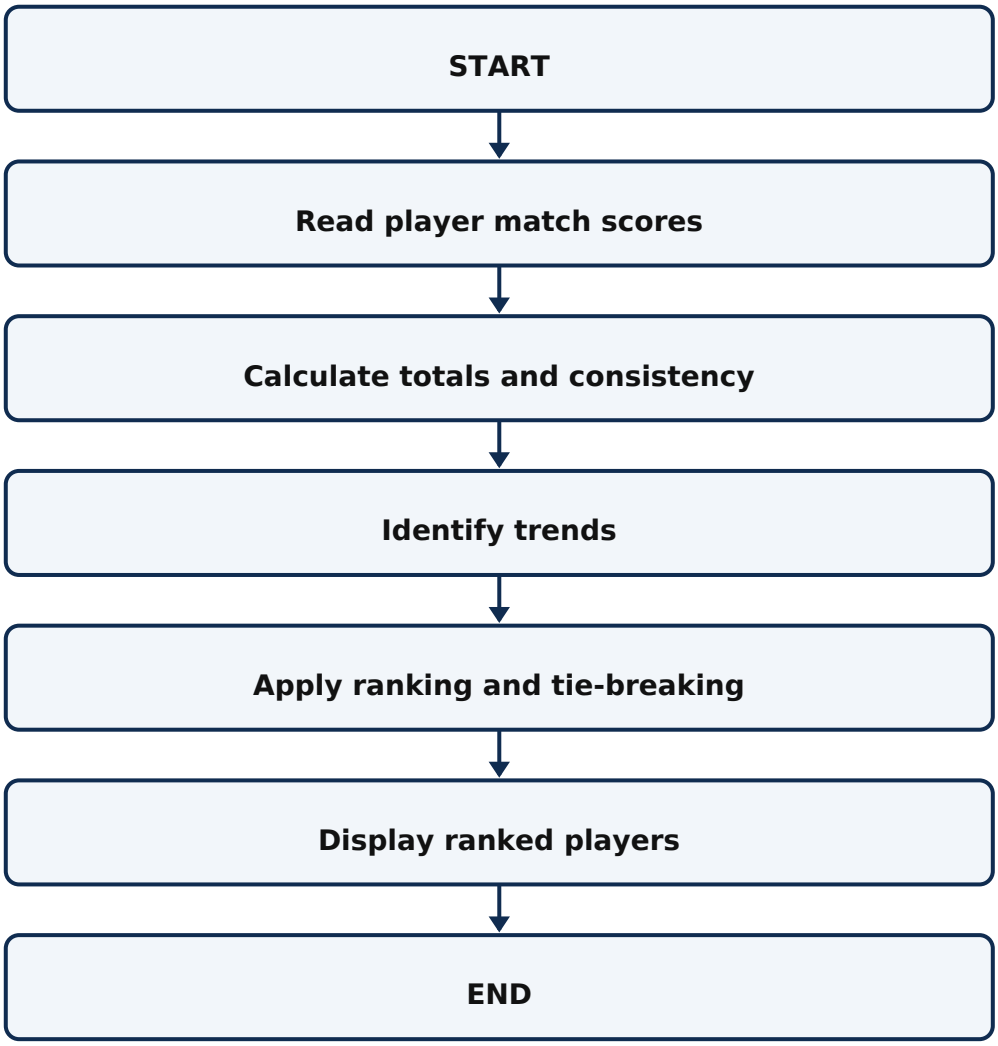
Assumptions

Higher total ranks first, then smaller range, then alphabetical name.

Algorithm

1. Read player match scores.
2. Calculate totals and consistency.
3. Identify trends.
4. Apply ranking and tie-breaking.
5. Display ranked players.

Flowchart (Connected Boxes)


START
Read player match scores
Calculate totals and consistency
Identify trends
Apply ranking and tie-breaking
Display ranked players
END

Pseudocode

```
BEGIN
    READ PLAYER MATCH SCORES
    CALCULATE TOTALS AND CONSISTENCY
    IDENTIFY TRENDS
    APPLY RANKING AND TIE-BREAKING
    DISPLAY RANKED PLAYERS
END
```

Python Source Code

```
"""Nested List Sports Performance Analysis"""
players = ['Asha', 'Bina', 'Chetan']
scores = [[10, 20, 30], [25, 25, 25], [30, 20, 10]]
ranking = []
for i in range(len(players)):
    s = scores[i]
    total = sum(s); avg = total / len(s); spread = max(s) - min(s)
    trend = 'Improving' if s[-1] > s[0] else ('Declining' if s[-1] < s[0] else 'Stable
    print(players[i], 'Total:', total, 'Average:', round(avg,2), 'Range:', spread, 'Tr
    ranking.append((players[i], total, spread))
print('Ranking:', [x[0] for x in sorted(ranking, key=lambda x: (-x[1], x[2], x[0]))])
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
Asha Total: 60 Average: 20.0 Range: 20 Trend: Improving
Bina Total: 75 Average: 25.0 Range: 0 Trend: Stable
Chetan Total: 60 Average: 20.0 Range: 20 Trend: Declining
Ranking: ['Bina', 'Asha', 'Chetan']
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.

70. Multi-Stage Scholarship Shortlisting

Problem Statement

Apply minimum academic eligibility, attendance requirements, income conditions, and ranking rules to select candidates within a fixed scholarship quota.

Objectives

Use Python lists, tuples, loops and conditions to process the case-study data, apply the required checks and display understandable results.

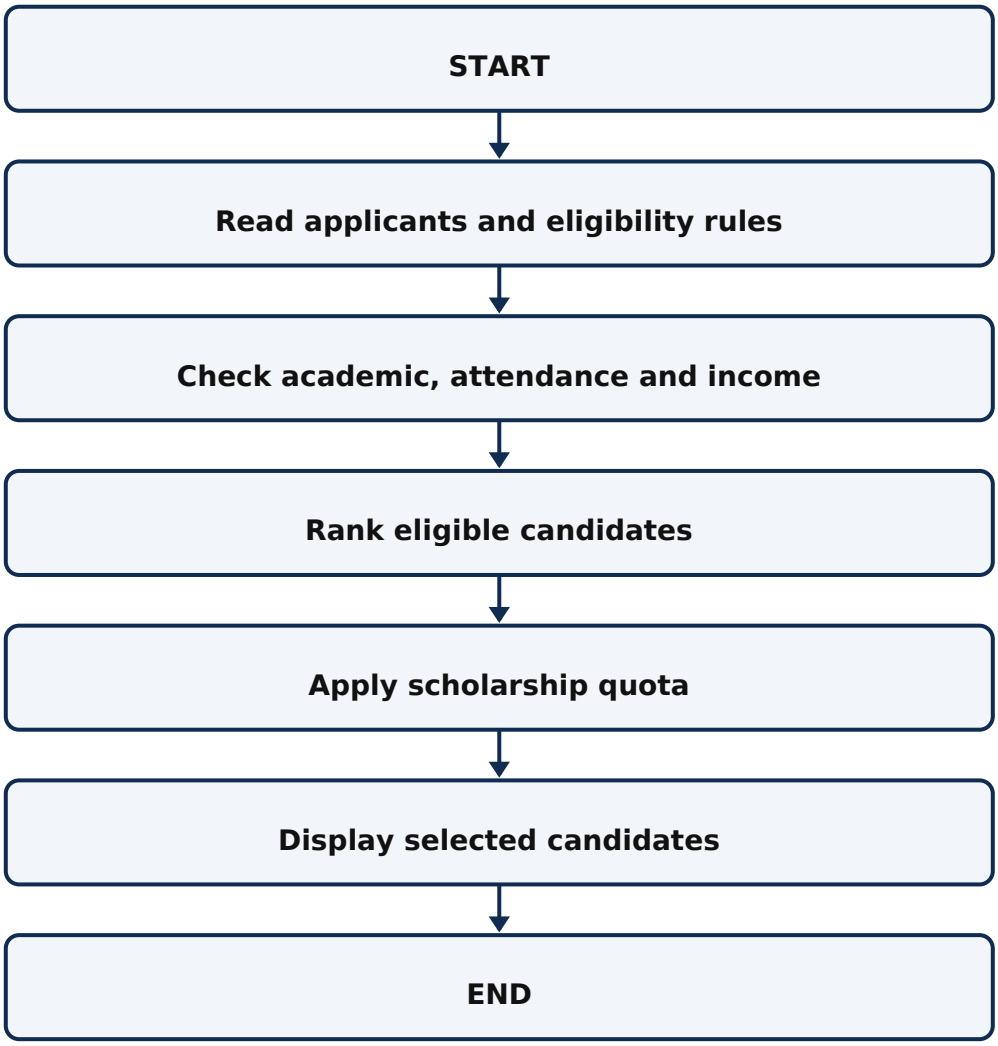
Assumptions

Minimum marks 75, attendance 80%, maximum income 200000; ties: higher attendance then lower income then name.

Algorithm

1. Read applicants and eligibility rules.
2. Check academic, attendance and income.
3. Rank eligible candidates.
4. Apply scholarship quota.
5. Display selected candidates.

Flowchart (Connected Boxes)


START
Read applicants and eligibility rules
Check academic, attendance and income
Rank eligible candidates
Apply scholarship quota
Display selected candidates
END

Pseudocode

```
BEGIN
    READ APPLICANTS AND ELIGIBILITY RULES
    CHECK ACADEMIC, ATTENDANCE AND INCOME
    RANK ELIGIBLE CANDIDATES
    APPLY SCHOLARSHIP QUOTA
    DISPLAY SELECTED CANDIDATES
END
```

Python Source Code

```
"""Multi-Stage Scholarship Shortlisting"""
candidates = [('Anu',90,95,150000), ('Bala',85,90,180000), ('Charan',95,70,100000), ('
quota = 2
eligible = []
for name, marks, attendance, income in candidates:
    if marks >= 75 and attendance >= 80 and income <= 200000:
        eligible.append((name, marks, attendance, income))
    else: print('Ineligible:', name)
eligible.sort(key=lambda x: (-x[1], -x[2], x[3], x[0]))
print('Selected:', [x[0] for x in eligible[:quota]])
print('Waitlisted:', [x[0] for x in eligible[quota:]])
```

Test Case 1

Execute the Python program using the sample dataset included in the source code.

Actual Output

```
Ineligible: Charan
Ineligible: Esha
Selected: ['Anu', 'Divya']
Waitlisted: ['Bala']
```

Additional Test Cases

Test 2: Change a sample input to a boundary value and verify the calculation or validation result.

Test 3: Introduce an invalid or duplicate record (where relevant) and check the displayed result.

Expected Result

The program follows the specified assumptions and displays calculated values or validation findings.

Conclusion

This case study demonstrates the use of beginner-level Python programming to process structured data and report results clearly.